



Создание проверка циклов в Claude Code с НАВЫКАМИ

Как превратить ручные
проверки в навыки, чтобы
Claude замкнул собственный
цикл обратной связи.

💡 Категория

[Код
Клода](#)

📦 Продукт

[Код
Клода](#)

📅 Дата

22 июля
2026
года

🕒 Время

чтения
5 мин

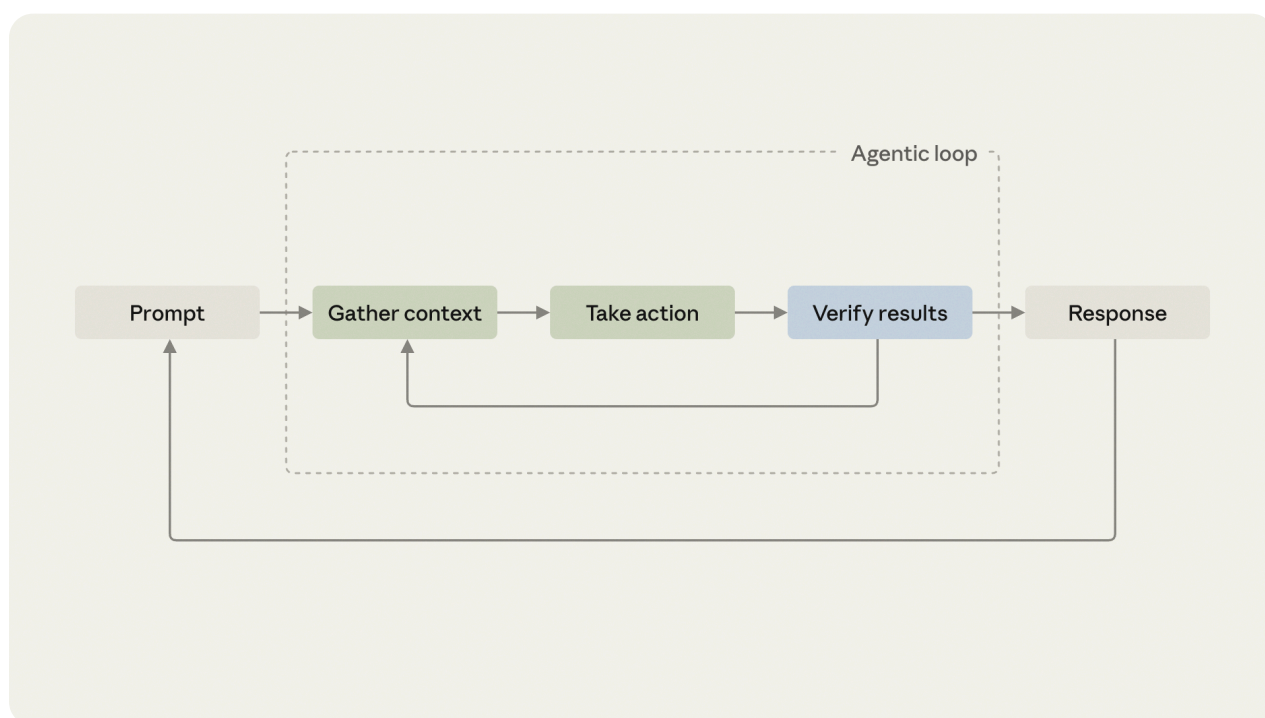
Поделиться

[Скопировать
ссылку](#)

Большинство сессий [агентного программирования](#) проходят по одному сценарию: вы запрашиваете изменение, Claude собирает контекст, выполняет действие, проверяет результаты и, при необходимости, возвращается к сбору дополнительного контекста.

определить, становится частью процесса ручной проверки функции.

Однако эти действия, выполняемые вручную, можно преобразовать в циклы проверки. В Claude Code цикл проверки — это итеративный процесс, в ходе которого Claude проверяет работу и пытается ее исправить.



Агентный цикл: 1. сбор информации, 2. принятие мер, 3. проверка результатов.

В этой статье мы расскажем о наиболее распространенных типах циклов верификации и покажем, какие из них мы используем в Anthropic. Затем мы покажем, как закодировать уже выполняемые вами ручные проверки в виде навыков, чтобы Claude мог замкнуть собственный цикл обратной связи, а вы могли заняться чем-то другим, пока он выполняет итерации.

Впервые слышите о циклах агентов? Начните с [раздела о начале работы с циклами](#).

двигаться дальше. В Claude Code циклы верификации можно упаковать в виде навыков, чтобы в каждом сеансе автоматически применялись одни и те же проверки, а не приходилось полагаться на то, что человек их запомнит.

Встроенные циклы верификации

Прежде чем приступать к разработке собственных циклов верификации, полезно ознакомиться со встроенной поддержкой различных циклов верификации в Claude. К общим функциям и подходам относятся:

- **/verify skill**: собирает, запускает и отслеживает изменения в вашем приложении.
- **Набор инструментов**: Claude выявляет коды ошибок и предупреждения любого предоставленного вами инструмента, например линтера, и реагирует на них. Рекомендуется указать в CLAUDE.md точные команды для сборки и тестирования, чтобы Claude не приходилось их угадывать.
- **Code Review (предварительная версия)**: управляемый многоагентный сервис, который автоматически проверяет PR в репозиториях, которые вы активировали. Вы можете исправить ошибку вручную и отправить изменения, либо закрыть цикл, оставив комментарий @claude к ошибке (если вы уже настроили GitHub Actions, см. ниже).
- **GitHub Actions**: определите задание, которое запускает Claude с помощью навыка проверки, и те же проверки, которые вы

репозитории и устранять нарушения.

- **Rubrics in Claude Managed Agents (бета):** управляемый агентский сервис, который позволяет проверять результаты по критериям с помощью отдельного агента-оценщика. При возникновении ошибок работа автоматически возвращается на доработку.

Writing verification loops

When you have an existing project and you find yourself making the same small corrections every time Claude implements a new feature for you, it's time to turn those steps into your own custom verification loop. The first step is to write down everything that you find yourself doing every time

The same goes if you're starting a new project and need to figure out how the project should behave. Write the best-practices version in plain English, the way you'd hand it to a new teammate on day one.

If you're struggling to articulate the verification check itself, ask Claude for best practices first and edit from there. Your version probably differs on a few specific points, and those differences are exactly what you want to capture.

Pro tip: The check doesn't have to be qualitative to belong here.

"Reject any migration that drops a column without a backfill step" is a deterministic rule no generic linter will catch but a project-specific one will. Anything you keep having to enforce by hand as a manual check qualifies for capture as a loop.

Make it a skill

The most common way to encode repetitive steps into a verification loop is to write it as a skill, and the fastest way to create a skill is to install the skill-creator plugin and let Claude interview you:

Example:

```
/skill-creator Create a skill for verifying frontend changes end-to-end. Interview me about my workflow.
```

You can also hand-write a skill by dropping a markdown file in `.claude/skills/` inside your project. The simplest possible verification skill is a few lines of frontmatter plus a body:

```
# .claude/skills/verify-log-hygiene/SKILL.md
---
name: verify-log-hygiene
description: Check that error logs include the request ID
and never
    include the request body. Use when the diff touches
error handling
    or logging.
allowed-tools: [Read, Edit, Grep]
---
Read the error-handling paths in the current diff.
```

```
Report each violation with file:line, then fix it: add the  
request ID  
where it's missing and strip the payload from the log call.
```

The full schema and the philosophy behind it are in our [complete guide to building skills](#).

Match the check to where it runs

The next thing to determine will be how the verification loop kicks off: standalone, embedded, chained, or tied to PR.

Standalone

You invoke it deliberately, after the artifact exists. A standalone skill earns its place for cross-cutting checks that don't apply every time: a pre-commit security scan, a pre-PR accessibility audit, license-header verification across a repo. Anything you want available across many workflows but don't want firing on every code change.

The cost is that each invocation is still a turn you have to remember to take. The signal that you've outgrown standalone is when you're running it after every change. At that point, the procedure has earned a permanent home: embed it or chain it.

Embedded

Fires automatically as part of the producing skill. The check belongs to one specific workflow, and the workflow now runs it without

```
# .claude/SKILLS/scaffold-component/SKILL.md
---
name: scaffold-component
description: Scaffold a new React component under
src/components/, including the component file, its co-
located test, and an index export. Use when the user asks to
create a new component.
allowed-tools: [Read, Write, Edit, Bash, Glob]
---
```

Scaffold a new React component

Given a component name (PascalCase), create the following under `src/components/<Name>/`:

1. `<<Name>.tsx`': function component with a typed props interface and a default export.
2. `<<Name>.test.tsx`': React Testing Library test that renders the component and asserts it mounts without throwing.
3. `index.ts`: re-export the default and any named exports.

Follow the patterns in `src/components/Button/` as the reference. Match the import alias style (`@/components/...`) used throughout the codebase.

code continues...

After creating the component file, run eslint on it and address any errors before reporting completion.

Embedded only works on skills you can edit: ones you wrote yourself, or ones installed at a project level where the SKILL.md file is under your control. Built-in skills and plugin-managed skills (the kind that get overwritten on update) are off-limits for this pattern; for those, chain instead.

Skip embedded for checks that span workflows; those want standalone, so you can invoke them from any context.

Chained

One skill calls another at its end, and several verified handoffs run end-to-end.

Members of Anthropic's Claude Code team use this pattern in their day-to-day: `/code-review` hunts for bugs, `/simplify` cleans up the diff, a `/verify` skill confirms end-to-end behavior, and a custom `/design` skill checks against guidelines in a DESIGN.md file if the change touched UI.

Chaining is also how you add verification to a skill you can't modify: build a custom wrapper skill that invokes the original, then invokes your verification skill, as depicted below:

```
# .claude/skills/safe-refactor/SKILL.md
Run /simplify on the current diff first.
When /simplify finishes, invoke /verify-no-public-
api-changes.
```

You can skip chaining when the steps are independent enough that you sometimes want to run one without the others; chaining trades flexibility for automation. Chained verification loops can increase token spend, so it's best to test these loops before deploying them broadly.

On every PR

Once the chain is solid for your own changes, the same procedure can run on every PR. A teammate's change passes the same gates yours did, whether they remembered to invoke the chain or not. The infrastructure is the same kind of thing as the chain you already wrote, one step further along: the same skills, the same rubrics, the same standards, applied without depending on the author's diligence.

This is where verification stops being personal infrastructure and becomes team infrastructure. The check you wrote down to save yourself two minutes a week is now saving everyone two minutes a week, on every change. Hold off on PR-wide gates while the chain is still in flux; every adjustment becomes a team-visible event.

Once you have the process down, you're ready to expand your loop engineering. The verification loop creation process is consistent, no matter what you're automating or in what environment:

1. Pick the manual follow-up you did most often this week.
2. Try out the built-in /verify skill first and see if it helps your process.
3. Write the procedure in plain English, the way you'd hand it to a new teammate on day one.
4. Hand it to skill-creator, or drop the markdown file in `.claude/skills/` yourself.
5. Invoke it on a new task and confirm the check runs as part of the output, iterate if needed.
6. Experiment with skill chaining to create an end-to-end verification flow.

The more you can encode for Claude to follow, the more often Claude's response will land closer to what you want on the very first try. The corrections you no longer have to fiddle with now free up your

attention to the individual and exclusive work that no skill can write down for you.

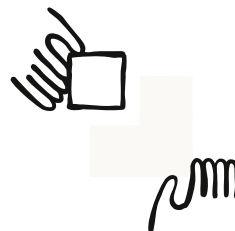
Related posts



Jul 24, 2026

Claude models explained: choosing the best model for your use case

 Enterprise AI



Jul 6, 2026

A field guide to Claude Fable 5: Finding your unknowns

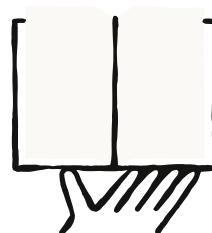
Claude Code



Jun 30, 2026

Loop engineering: Getting started with loops

Claude Code



Jul 24, 2026

The new rules of context engineering for Claude 5 generation models

Claude Code

operates with Claude

[See pricing](#)[Contact sales](#)

Get the developer newsletter

Product updates, how-tos, community spotlights, and more. Delivered monthly to your inbox.



Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.



[@Claude](#)[Claude Design](#)[Claude Science](#)[Claude Security](#)[Download app](#)[Pricing](#)[Log in](#)[Features](#)[Claude for Chrome](#)[Claude for](#)[Microsoft 365](#)[Skills](#)[Models](#)[Mythos](#)[Fable](#)[Opus](#)[Sonnet](#)[Haiku](#)[Enterprise](#)[Financial services](#)[Government](#)[Healthcare](#)[Higher education](#)[K-12 teachers](#)[Legal](#)[Life sciences](#)[Nonprofits](#)[Small business](#)[Claude Platform](#)[Overview](#)[Developer docs](#)[Pricing](#)[Ecosystem](#)[Marketplace](#)[Claude on AWS](#)[Google Cloud](#)[Microsoft Foundry](#)[Regional compliance](#)[Console login](#)[Courses](#)[Customer stories](#)[Engineering at Anthropic](#)[Events](#)[Plugins](#)[Powered by Claude](#)[Service partners](#)[Tutorials](#)[Use cases](#)[Company](#)[Anthropic](#)[Careers](#)[Policy](#)[Economic Futures](#)[Research](#)[News](#)[Policy on the AI Exponential](#)[Responsible Scaling Policy](#)[Security and compliance](#)[Transparency](#)[Availability](#)[Status](#)[Support center](#)[Terms and policies](#)[Privacy choices](#)[Privacy policy](#)[Responsible disclosure policy](#)[Terms of service: Commercial](#)[Terms of service: Consumer](#)[Terms of Service: US K-12](#)[Data Processing](#)[Agreement: US K-12](#)[Usage policy](#)

ANTHROPIC

© 2026 ANTHROPIC PBC. SERVICES IN THE EU ARE PROVIDED BY ANTHROPIC IRELAND LIMITED.

[English \(US\) ▾](#)

